

Building Low Latency Applications with C++

Chapter 8: Processing Market Data and Sending Orders to the Exchange in C++

Week 6: Client connectivity

Qishi C++ Study Group

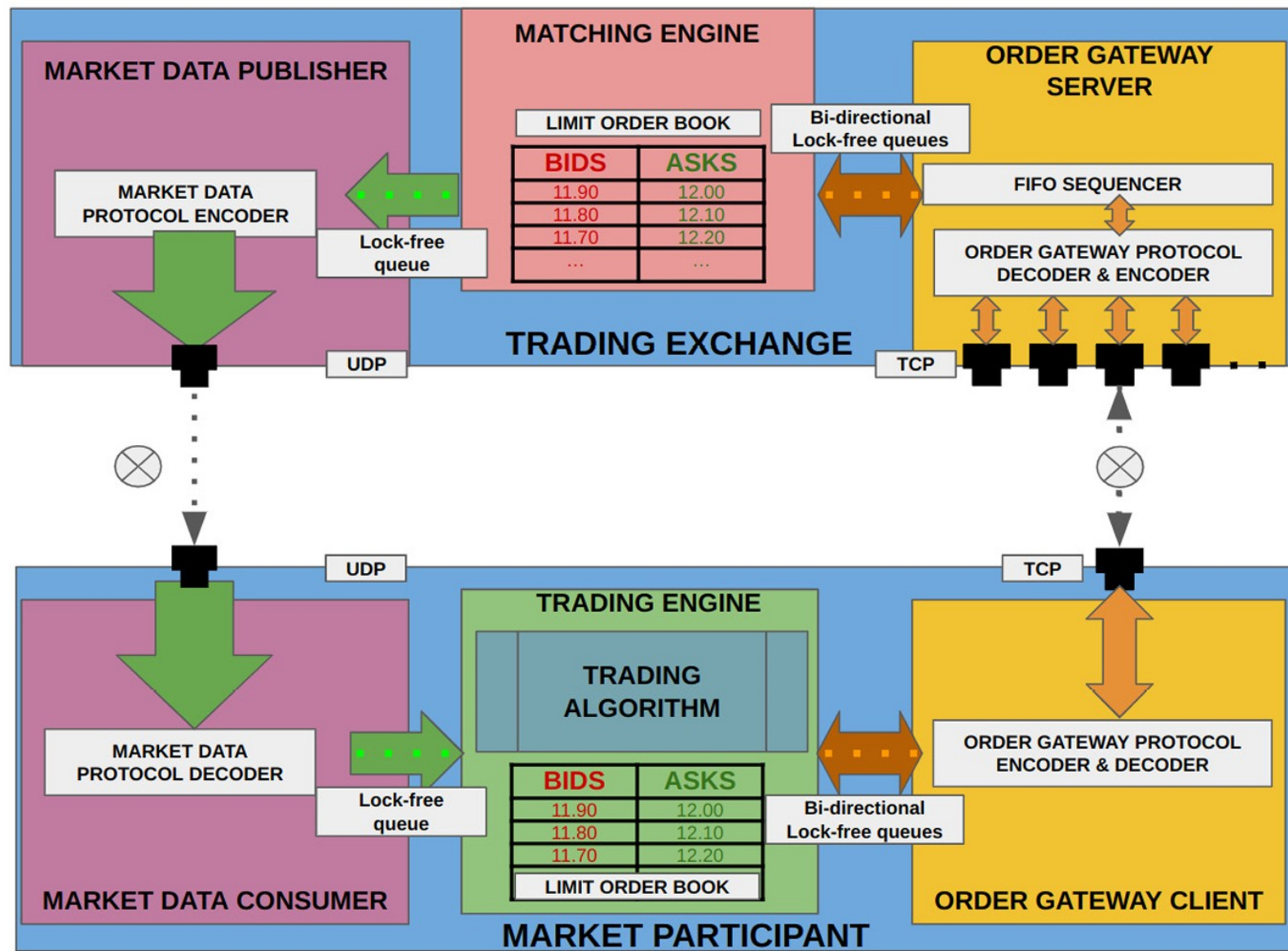


Figure 5.1 – Topology of a simple electronic trading ecosystem

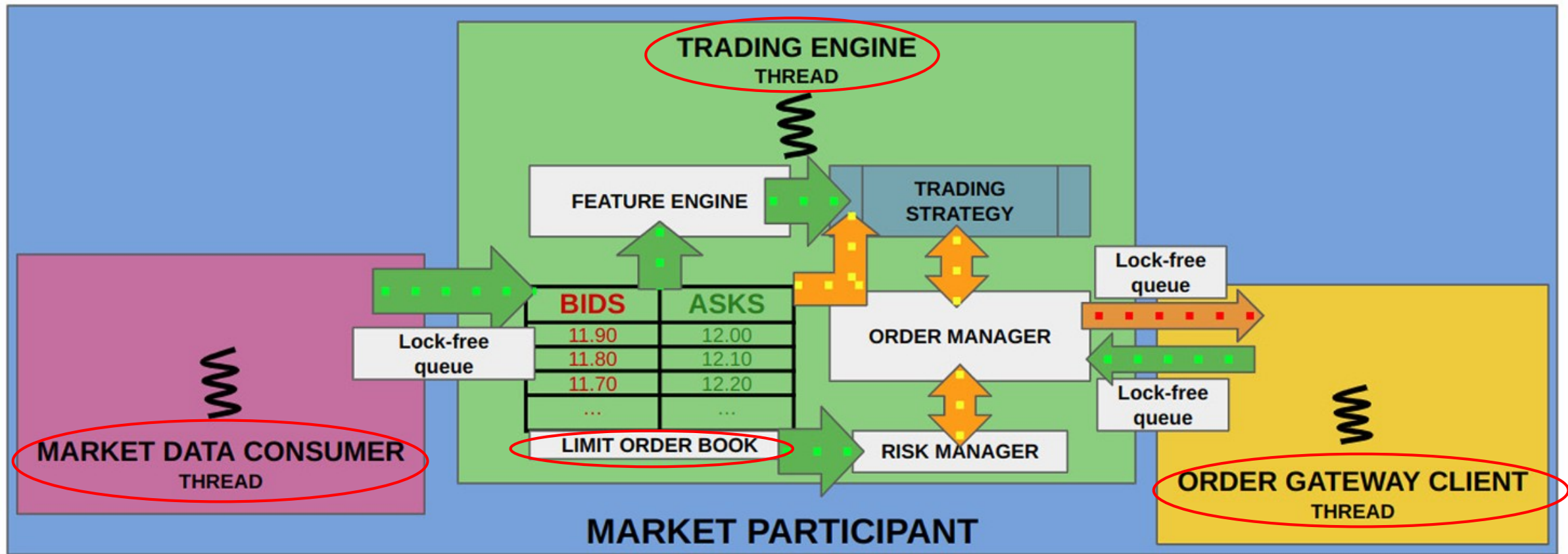


Figure 5.9 – Design of our trading strategy framework

Part 1: Trading Engine and OrderBook

Chapter8/trading/strategy/trade_engine.h

```
class TradeEngine {
private:
    Exchange::ClientRequestLFQueue *outgoing_ogw_requests_ = nullptr;
    Exchange::ClientResponseLFQueue *incoming_ogw_responses_ = nullptr;
    Exchange::MEMarketUpdateLFQueue *incoming_md_updates_ = nullptr;

    /// Hash map container from TickerId -> MarketOrderBook.
    MarketOrderBookHashMap ticker_order_book_;
}
```

Chapter8/trading/strategy/trade_engine.cpp

```
/// trade_engine.cpp
auto TradeEngine::run() noexcept->void {
    while (run_) {
        for (auto client_response=incoming_ogw_responses_->getNextToRead(); client_response;
client_response=incoming_ogw_responses_->getNextToRead()) {
            // process order update
        }
        for (auto market_update = incoming_md_updates_->getNextToRead(); market_update;
market_update=incoming_md_updates_->getNextToRead()) {
            // process market data update

            ticker_order_book_[market_update->ticker_id_->onMarketUpdate(market_update);
            incoming_md_updates_->updateReadIndex();
            last_event_time_=Common::getCurrentNanos();
        }
    }
}
```

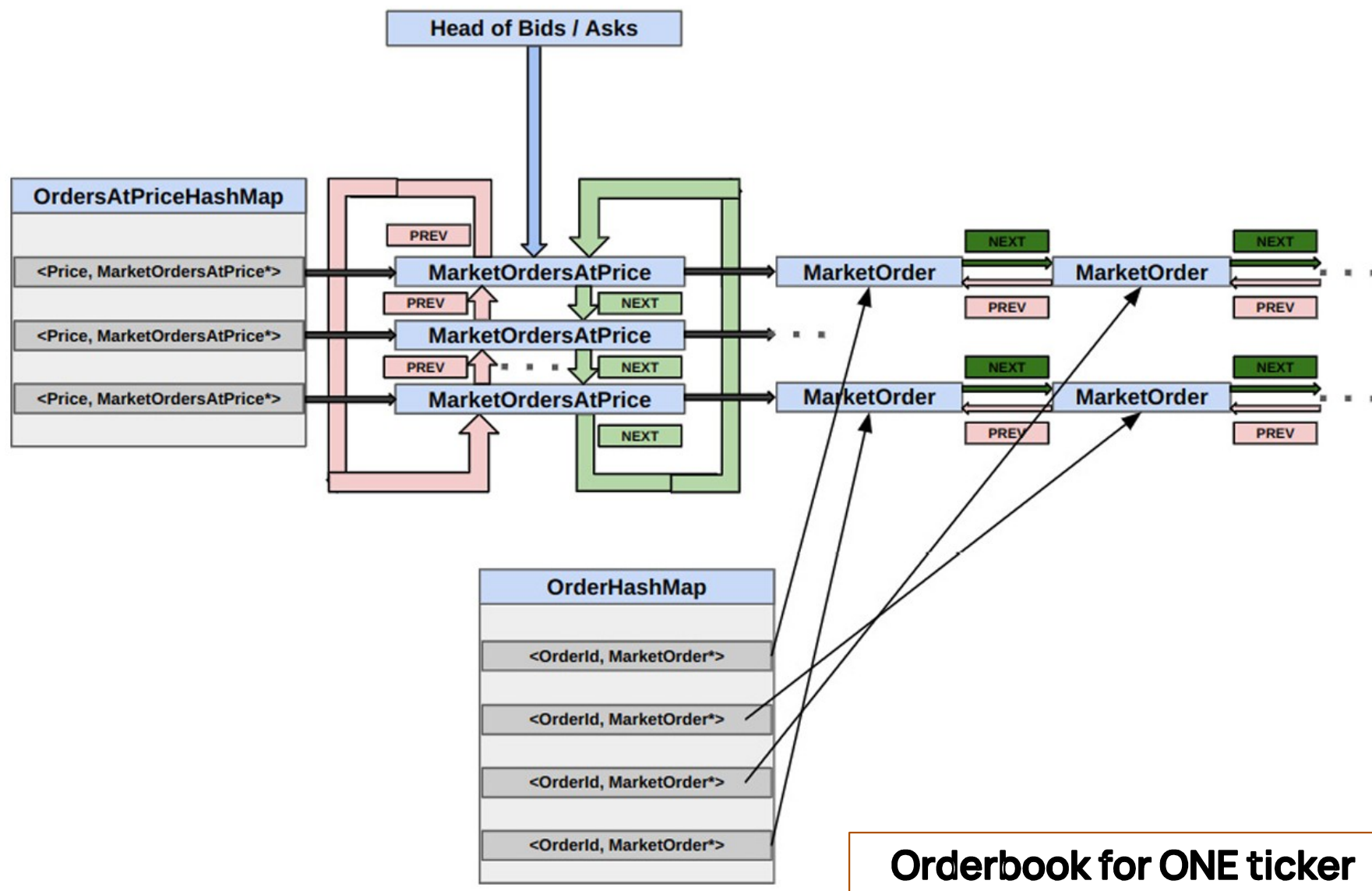


Figure 8.3 – Architecture of the limit order book in the market participant's trading engine

Chapter8/exchange/matcher/me_order.h

```
struct MEOrder {  
  
    TickerId ticker_id_;  
    ClientId client_id_;  
    OrderId client_order_id_;  
    OrderId market_order_id_;  
    Side side_;  
    Price price_;  
    Qty qty_;  
    Priority priority_;  
  
    MEOrder *prev_order_;  
    MEOrder *next_order_;  
};
```

Chapter8/trading/strategy/market_order.h

```
struct MarketOrder {  
  
    OrderId order_id_;  
    Side side_;  
    Price price_;  
    Qty qty_;  
    Priority priority_;  
  
    MarketOrder*prev_order_;  
    MarketOrder*next_order_;  
};
```

```
class MarketOrderBook final {
private:
    const TickerId ticker_id_;

    TradeEngine *trade_engine_= nullptr;

    OrderHashMap oid_to_order_;

    MemPool<MarketOrdersAtPrice> orders_at_price_pool_;

    MarketOrdersAtPrice *bids_by_price_=nullptr;
    MarketOrdersAtPrice *asks_by_price_=nullptr;

    OrdersAtPriceHashMap price_orders_at_price_;

    MemPool<MarketOrder> order_pool_;

    BBO bbo_;

    /// Hash map from TickerId -> MarketOrderBook.
    typedef std::array<MarketOrderBook*, ME_MAX_TICKERS> MarketOrderBookHashMap;

}
```


💡 BBO (Best Bid and Offer (Best Ask))

➤ Market price reference

The mid price is often computed : $(\text{best bid} + \text{best ask}) / 2$
weighted average price

➤ Liquidity signal

$\text{spread} = (\text{best ask}) - (\text{best bid})$

- Tight spread → liquid market
- Wide spread → illiquid / volatile

➤ Risk Management

open Profit and Loss (PnL)

➤ Trading decisions and other components

FeatureEngine, PositionKeeper, LiquidityTaker, MarketMaker

Order book:

Bids (Buy)	Asks (Sell)
-----	-----
100.10	100.20
100.05	100.25
100.00	100.30

👉 BBO =

- Best Bid = 100.10
- Best Ask = 100.20

Chapter8/trading/strategy/market_order.h

```
struct BBO {  
    Price bid_price_,  
          ask_price_;  
    Qty  bid_qty_,  
          ask_qty_;  
};
```

Chapter8/trading/strategy/market_order_book.h

```
/// Update the BBO abstraction, the two boolean parameters represent if the buy or the
/// sell (or both) sides or both need to be updated.
auto updateBBO(bool update_bid, bool update_ask) noexcept {
    if(update_bid) {
        if(bids_by_price_) {

            bbo_.bid_price_=bids_by_price_>price_;

            bbo_.bid_qty_=bids_by_price_>first_mkt_order_>qty_;

            for(auto order =bids_by_price_>first_mkt_order_>next_order_;
                order!=bids_by_price_>first_mkt_order_;
                order=order->next_order_)
                bbo_.bid_qty_+=order->qty_;
        }
        else {
            bbo_.bid_price_=Price_INVALID;
            bbo_.bid_qty_=Qty_INVALID;
        }
    }

    if(update_ask) {
        ...
    }
}
```

```

auto MarketOrderBook::onMarketUpdate(const Exchange::MEMarketUpdate*market_update) noexcept->void {
    const auto bid_updated= (bids_by_price_ && market_update->side_==Side::BUY
                             && market_update->price_>=bids_by_price_->price_);

    const auto ask_updated= (asks_by_price_ && market_update->side_==Side::SELL
                             && market_update->price_<=asks_by_price_->price_);

    switch (market_update->type_) {
        case Exchange::MarketUpdateType::ADD {...}
        case Exchange::MarketUpdateType::MODIFY: {...}
        case Exchange::MarketUpdateType::CANCEL: {...}
        case Exchange::MarketUpdateType::TRADE: {...}
        case Exchange::MarketUpdateType::CLEAR: {
            // Clear the full order book and deallocate MarketOrdersAtPrice and MarketOrder objects.
            for (auto&order: oid_to_order_) {if (order) order_pool_.deallocate(order);}
            oid_to_order_.fill(nullptr);
            if(bids_by_price_) {for(...) orders_at_price_pool_.deallocate(...);}
            if(asks_by_price_) {for(...) orders_at_price_pool_.deallocate(...);}
            bids_by_price_=asks_by_price_=nullptr;
            price_orders_at_price_.fill(nullptr);
        }
        break;
        case Exchange::MarketUpdateType::INVALID:
        case Exchange::MarketUpdateType::SNAPSHOT_START:
        case Exchange::MarketUpdateType::SNAPSHOT_END:
            break;
    }
    updateBB0(bid_updated, ask_updated);

    trade_engine_->onOrderBookUpdate(market_update->ticker_id_, market_update->price_,
                                     market_update->side_, this);
}

```

Part 2: Order Gateway

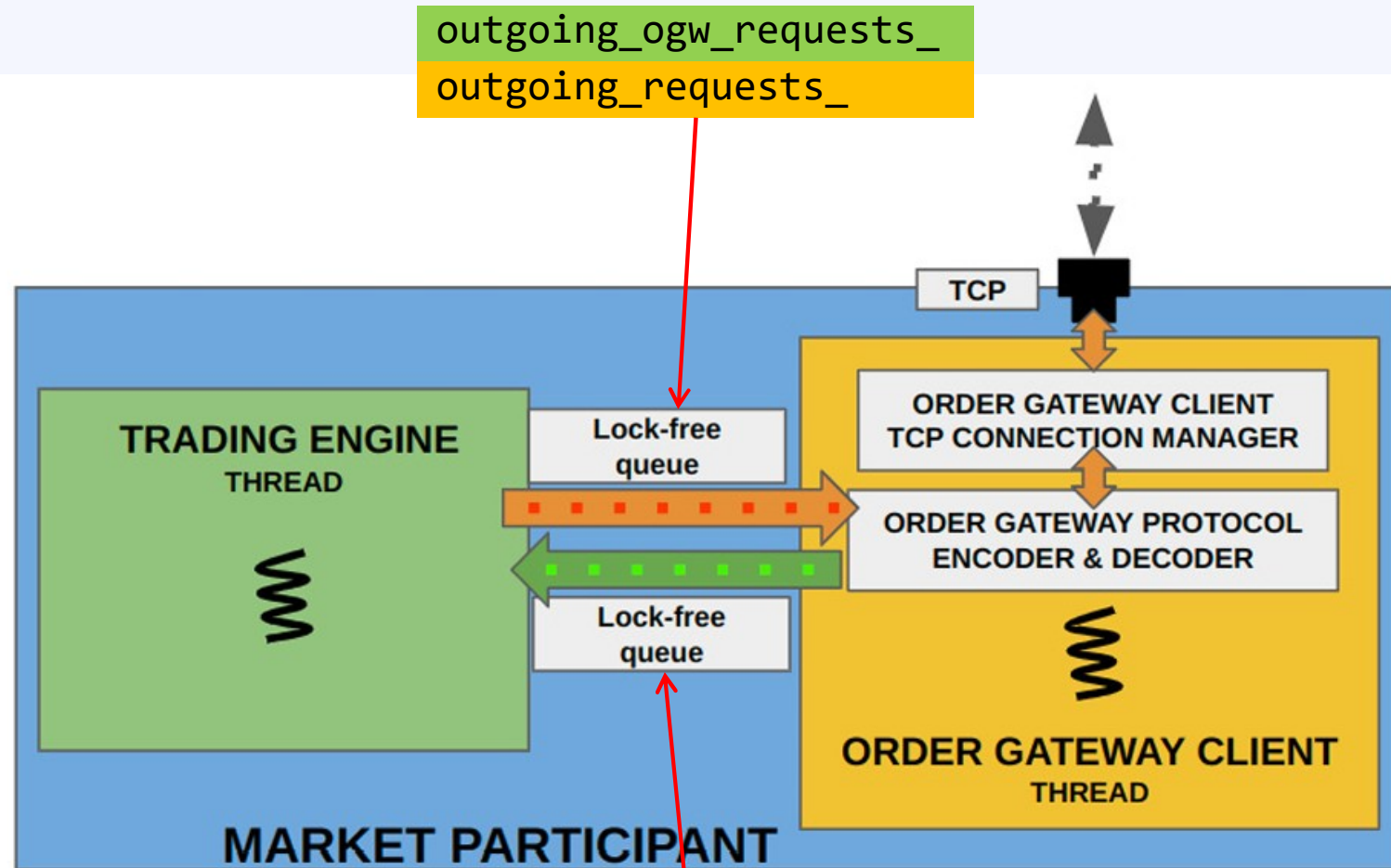
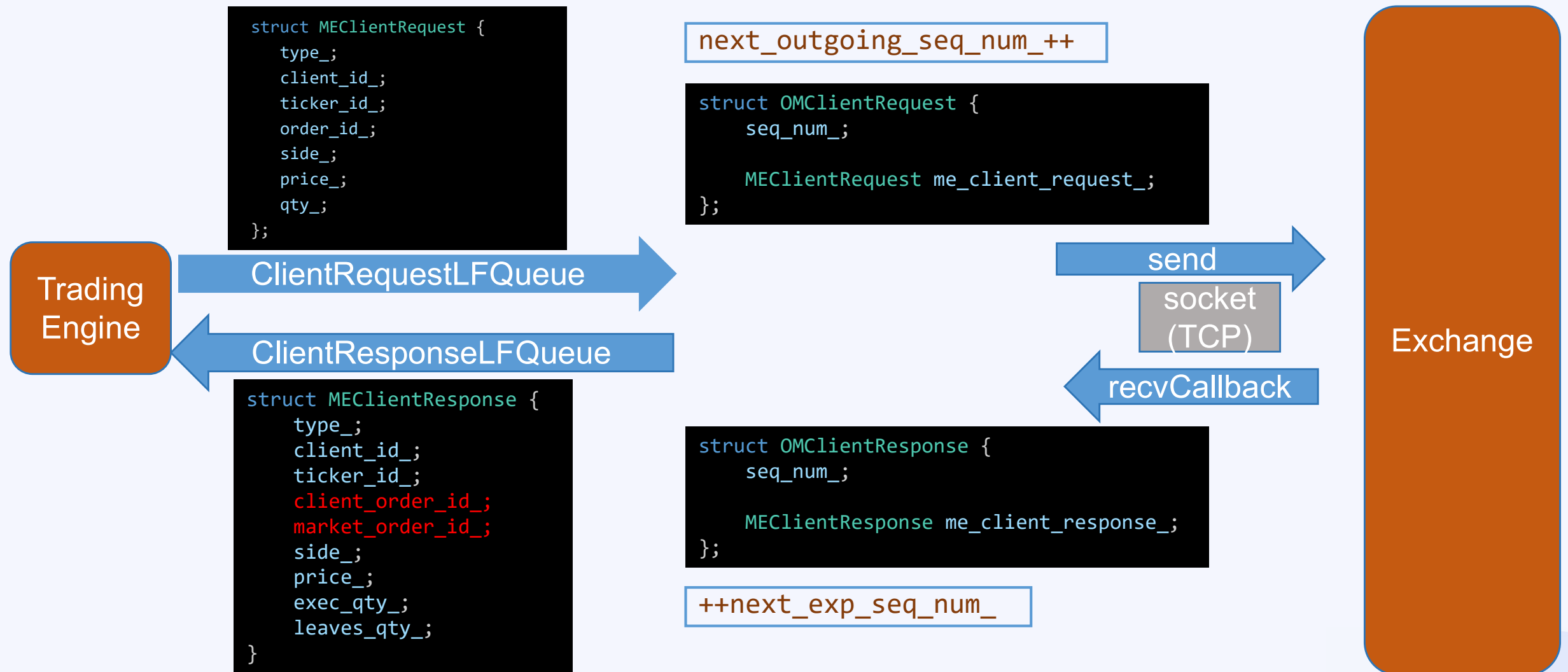


Figure 8.4 – Diagram presenting the order gateway client component inside the client's trading infrastructure



Chapter8/trading/order_gw/order_gateway.h

```
class OrderGateway {
private:
    const ClientId client_id_;

    /// Exchange's order server's TCP server address.
    std::string ip_;
    const std::string iface_;
    const int port_=0;

    /// consume client requests from the trade engine and forward them to the exchange
    Exchange::ClientRequestLFQueue* outgoing_requests_=nullptr;
    /// write client responses which read and processed from the ex to trade engine.
    Exchange::ClientResponseLFQueue* incoming_responses_=nullptr;

    volatile bool run_ = false;

    /// Sequence numbers to track the sequence number to set on outgoing client requests
    size_t next_outgoing_seq_num_=1;
    ///Sequence number expected on incoming client responses.
    size_t next_exp_seq_num_=1;

    /// TCP connection to the exchange's order server.
    Common::TCPSocket tcp_socket_;
private:
    auto recvCallback(TCPSocket*socket, Nanosrx_time) noexcept->void;
};
```

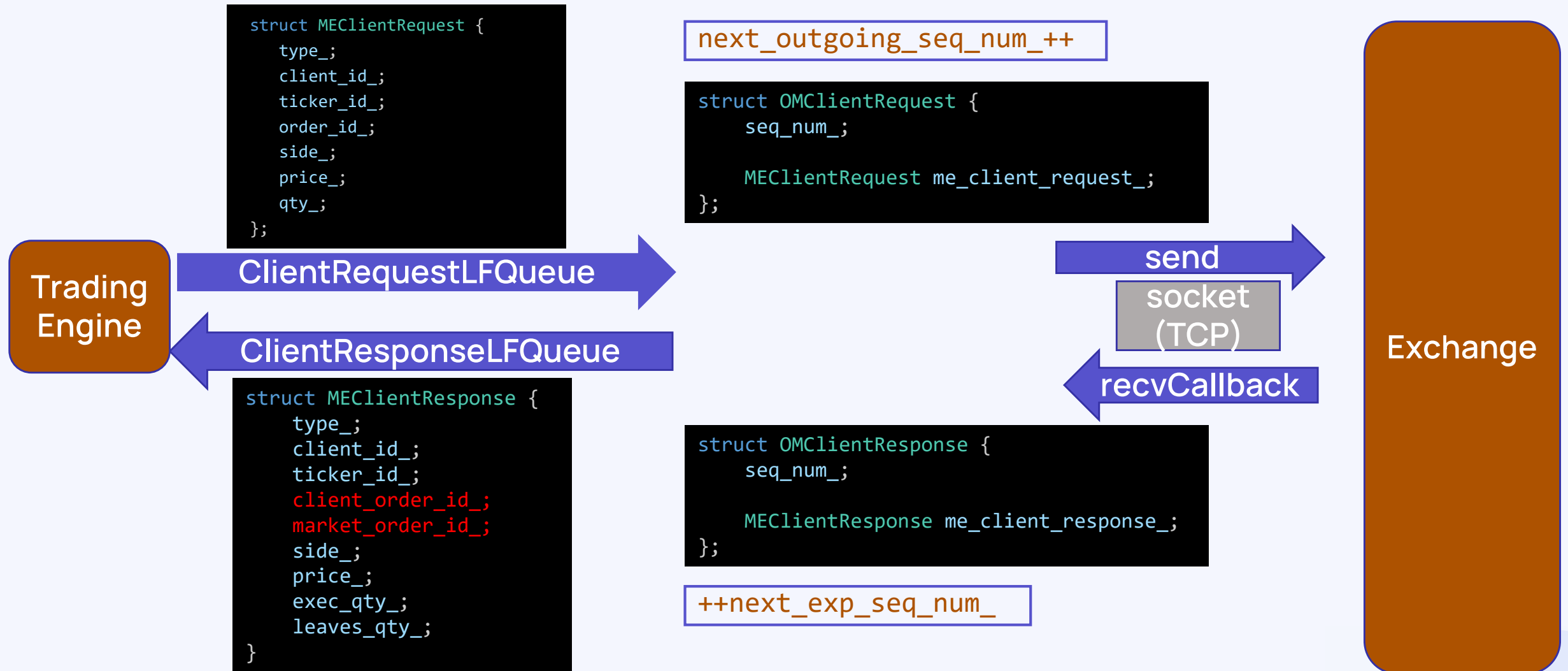
Chapter8/trading/order_gw/order_gateway.cpp

```
auto OrderGateway::run() noexcept->void {
    while (run_) {
        tcp_socket_.sendAndRecv();

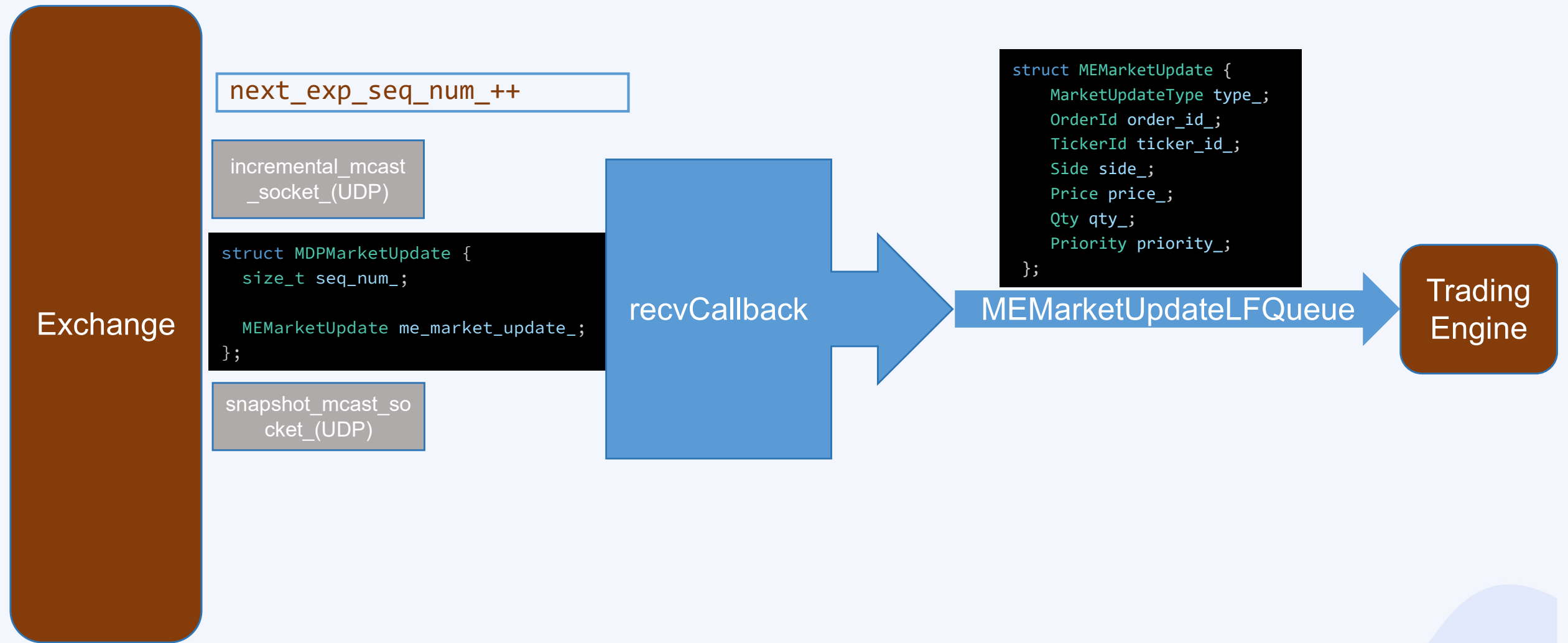
        for(auto client_request=outgoing_requests_->getNextToRead(); client_request; client_request=outgoing_requests_->getNextToRead()) {

            tcp_socket_.send(&next_outgoing_seq_num_, sizeof(next_outgoing_seq_num_));
            tcp_socket_.send(client_request, sizeof(Exchange::MEClientRequest));
            outgoing_requests_->updateReadIndex();
            next_outgoing_seq_num_++;
        }
    }
}

auto OrderGateway::recvCallback(TCP Socket*socket, Nanosrx_time) noexcept->void {
    if (socket->next_rcv_valid_index_>=sizeof(Exchange::OMClientResponse)) {
        size_t i=0;
        for (; i+sizeof(Exchange::OMClientResponse)<=socket->next_rcv_valid_index_;i+=sizeof(Exchange::OMClientResponse)){
            auto response = reinterpret_cast<const Exchange::OMClientResponse*>(socket->inbound_data_.data() +i);
            // ... check response client id and seq_num = next_exp_seq_num_...
            ++next_exp_seq_num_;
            auto next_write = incoming_responses_->getNextToWriteTo();
            *next_write = std::move(response->me_client_response_);
            incoming_responses_->updateWriteIndex();
        }
        memcpy(socket->inbound_data_.data(), socket->inbound_data_.data() +i, socket->next_rcv_valid_index_-i);
        socket->next_rcv_valid_index_-=i;
    }
}
```

Part 3: Market Data Consumer



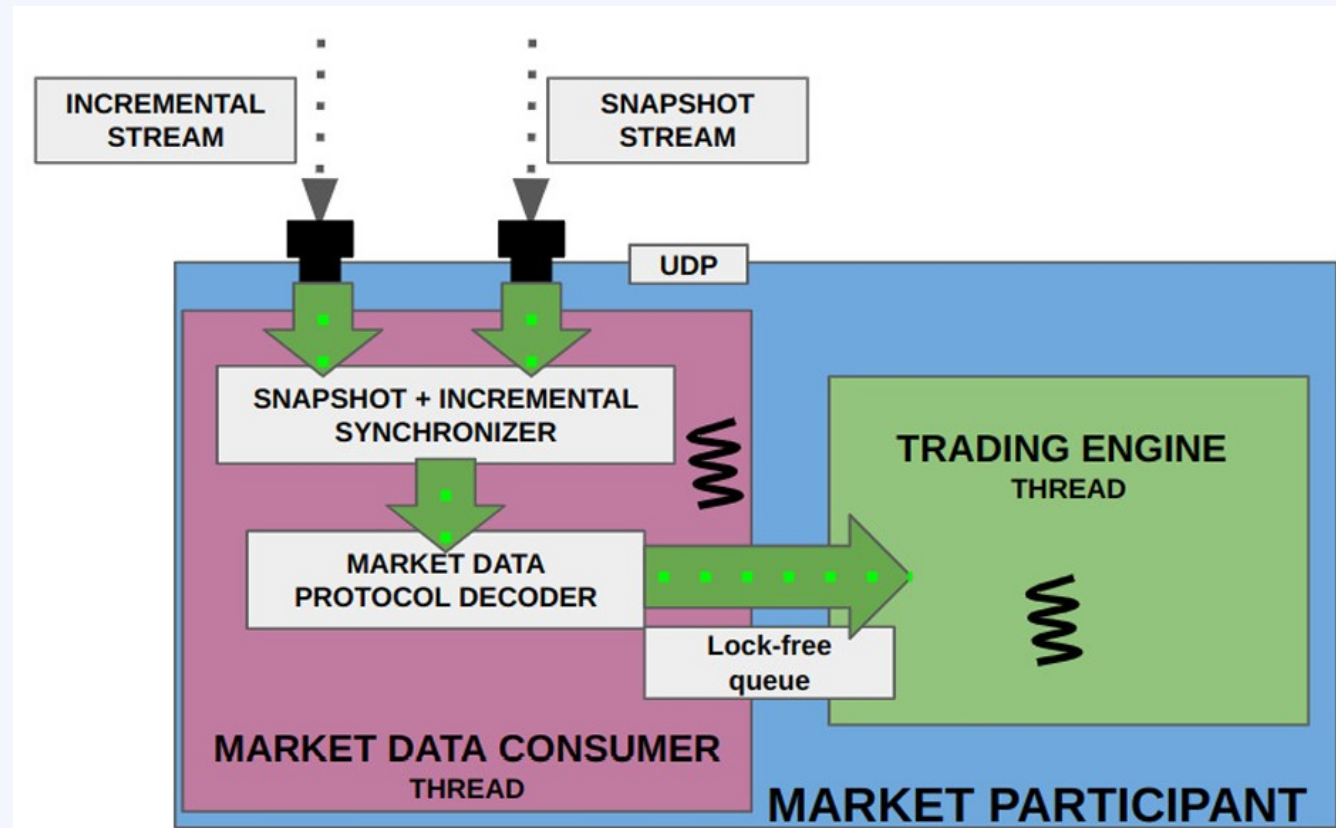


Figure 8.1 – An overview of the market data consumer component and its sub-components

Chapter8/trading/market_data/market_data_consumer.h

```
class MarketDataConsumer {  
  
    size_t next_exp_inc_seq_num_=1;  
  
    Exchange::MEMarketUpdateLFQueue* incoming_md_updates_;  
  
    Common::McastSocket incremental_mcast_socket_,  
                        snapshot_mcast_socket_;  
  
    bool in_recovery_=false;  
  
    const std::string iface_, snapshot_ip_;  
    const int snapshot_port_;  
  
    typedef std::map<size_t, Exchange::MEMarketUpdate> QueuedMarketUpdates;  
  
    QueuedMarketUpdates snapshot_queued_msgs_,  
                        incremental_queued_msgs_;  
  
};
```

Standard Template Library (STL)
std::map (Red Black Tree)

- easier to iterate
- insertion $O(\log(N))$
- both **seq** and **MEMarketUpdate**

Chapter8/trading/market_data/market_data_consumer.cpp

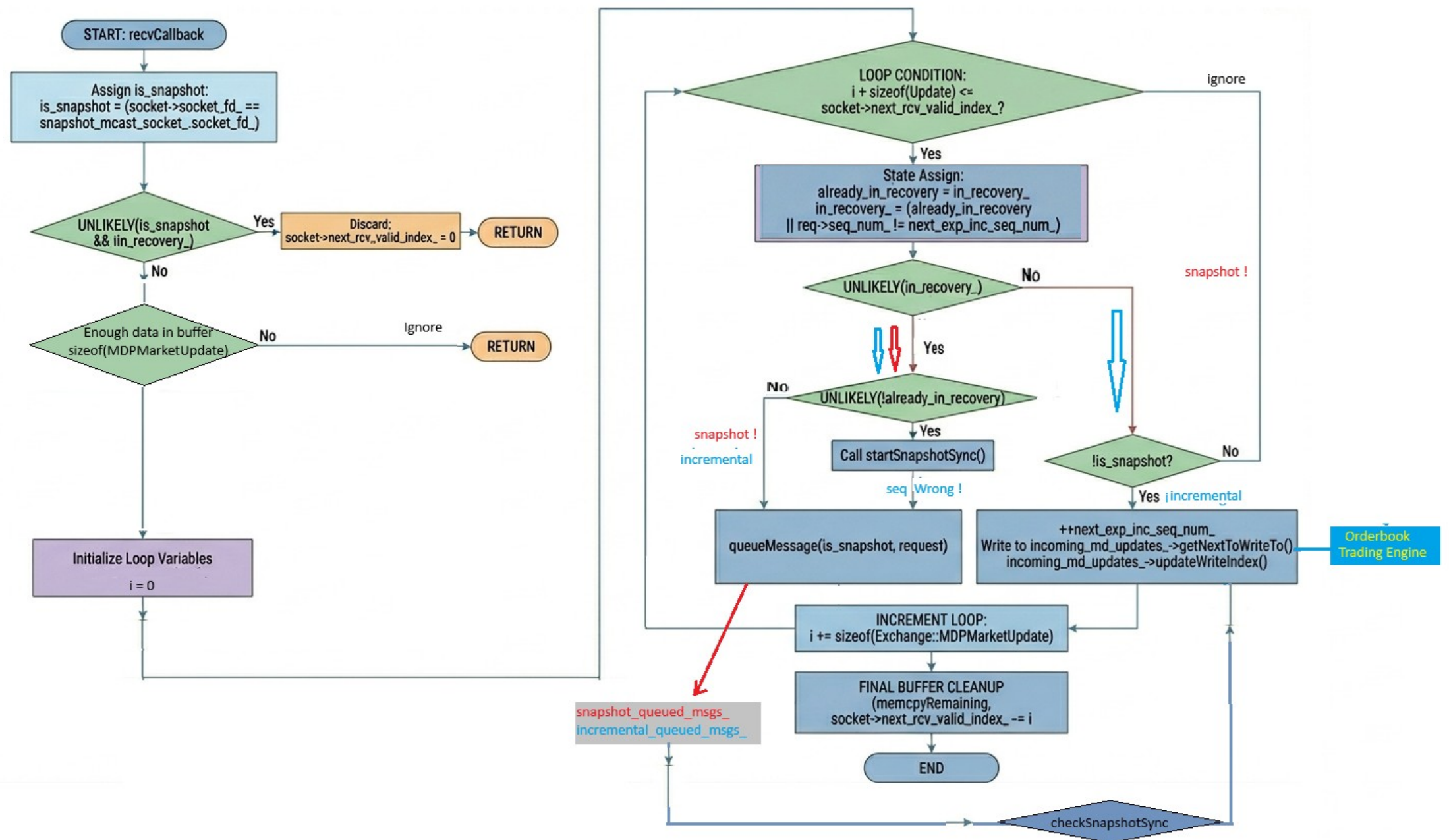
```
MarketDataConsumer::MarketDataConsumer(...)
: incoming_md_updates_(market_updates), run_(false),
  logger_("trading_market_data_consumer_"+std::to_string(client_id) + ".log"),
  incremental_mcast_socket_(logger_), snapshot_mcast_socket_(logger_),
  iface_(iface), snapshot_ip_(snapshot_ip), snapshot_port_(snapshot_port) {

    auto recv_callback= [this](auto socket) {
        recvCallback(socket);
    };

    incremental_mcast_socket_.recv_callback_ = recv_callback;
    ASSERT(incremental_mcast_socket_.init(incremental_ip, iface, incremental_port, /*is_lsn*/true) >=0, "err");
    ASSERT(incremental_mcast_socket_.join(incremental_ip), "Join failed on:");

    snapshot_mcast_socket_.recv_callback_ = recv_callback;
}

auto MarketDataConsumer::run() noexcept->void {
    while (run_) {
        incremental_mcast_socket_.sendAndRecv();
        snapshot_mcast_socket_.sendAndRecv();
    }
}
```



Chapter8/trading/market_data/market_data_consumer.cpp

```
auto MarketDataConsumer::startSnapshotSync() -> void {
    snapshot_queued_msgs_.clear();
    incremental_queued_msgs_.clear();

    ASSERT(snapshot_mcast_socket_.init(snapshot_ip_, iface_, snapshot_port_, /*is_lsn*/true) >=0, "error:");
    ASSERT(snapshot_mcast_socket_.join(snapshot_ip_), "Join failed on:");
}

auto MarketDataConsumer::queueMessage(bool is_snapshot, const Exchange::MDPMarketUpdate* request) {
    if (is_snapshot) {
        if (snapshot_queued_msgs_.find(request->seq_num_) != snapshot_queued_msgs_.end()) {
            snapshot_queued_msgs_.clear();
        }

        snapshot_queued_msgs_[request->seq_num_] = request->me_market_update_;
    } else {
        incremental_queued_msgs_[request->seq_num_] = request->me_market_update_;
    }

    checkSnapshotSync();
}
```


CheckSnapshotSync

1. The logic is to queue up messages received on the **snapshot** and **incremental** market data streams.
2. Then, when we receive `MarketUpdateType::SNAPSHOT_END`, we make sure that no messages were dropped on the snapshot market data stream by checking that there is no gap in the **sequence number** field on the snapshot messages.
3. Then, we inspect the queued market updates from the **incremental** data stream and check to see if we have messages following the last message that was used to synthesize this round of snapshot messages.
We do this by checking if we have market updates in the **incremental** queue starting with a **sequence number** equal to the **OrderId + 1** value from the **SNAPSHOT_END** message in the **snapshot queue**.
4. Finally, we check to make sure that from that point on in the incremental queued messages, we do not have another gap

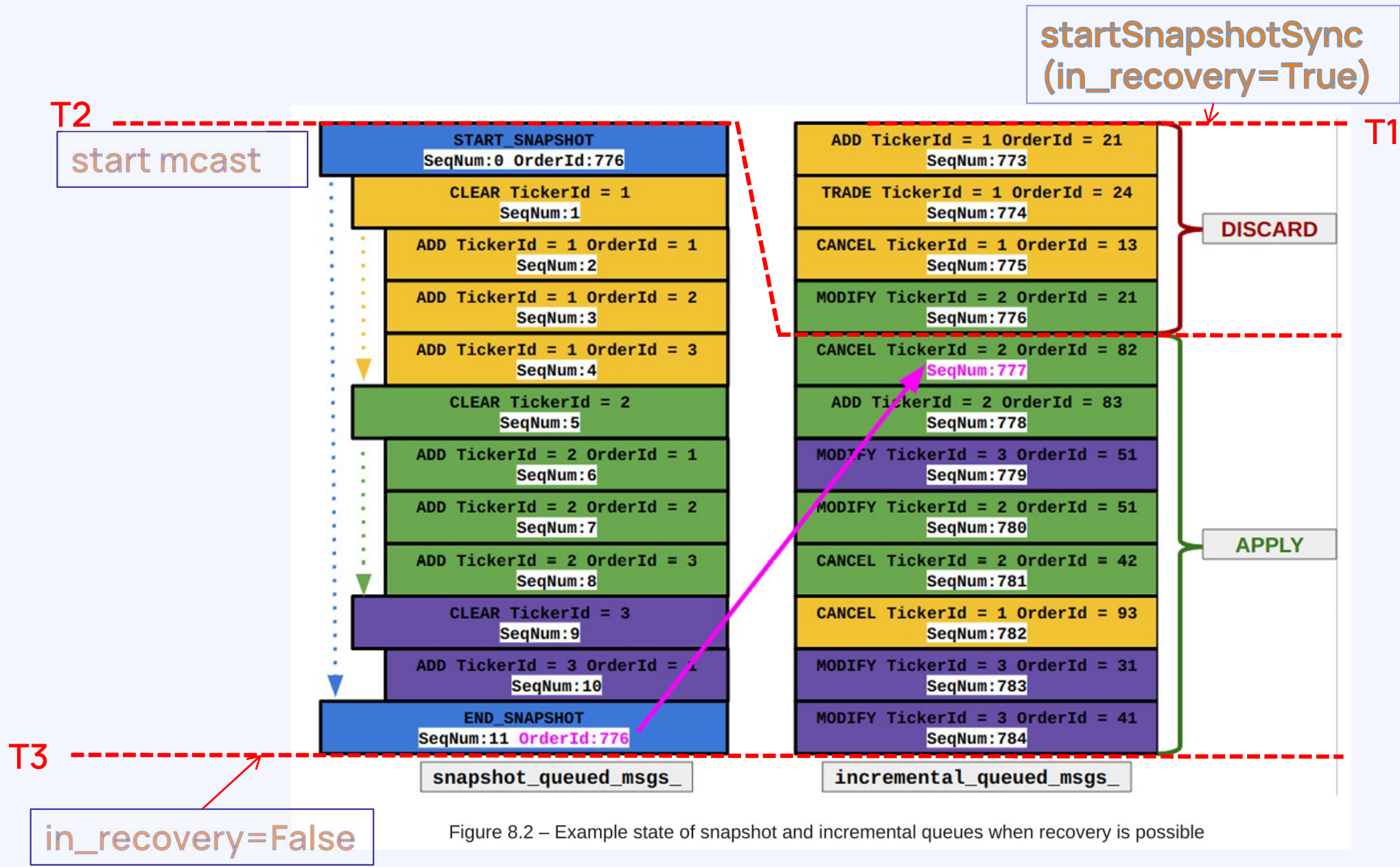
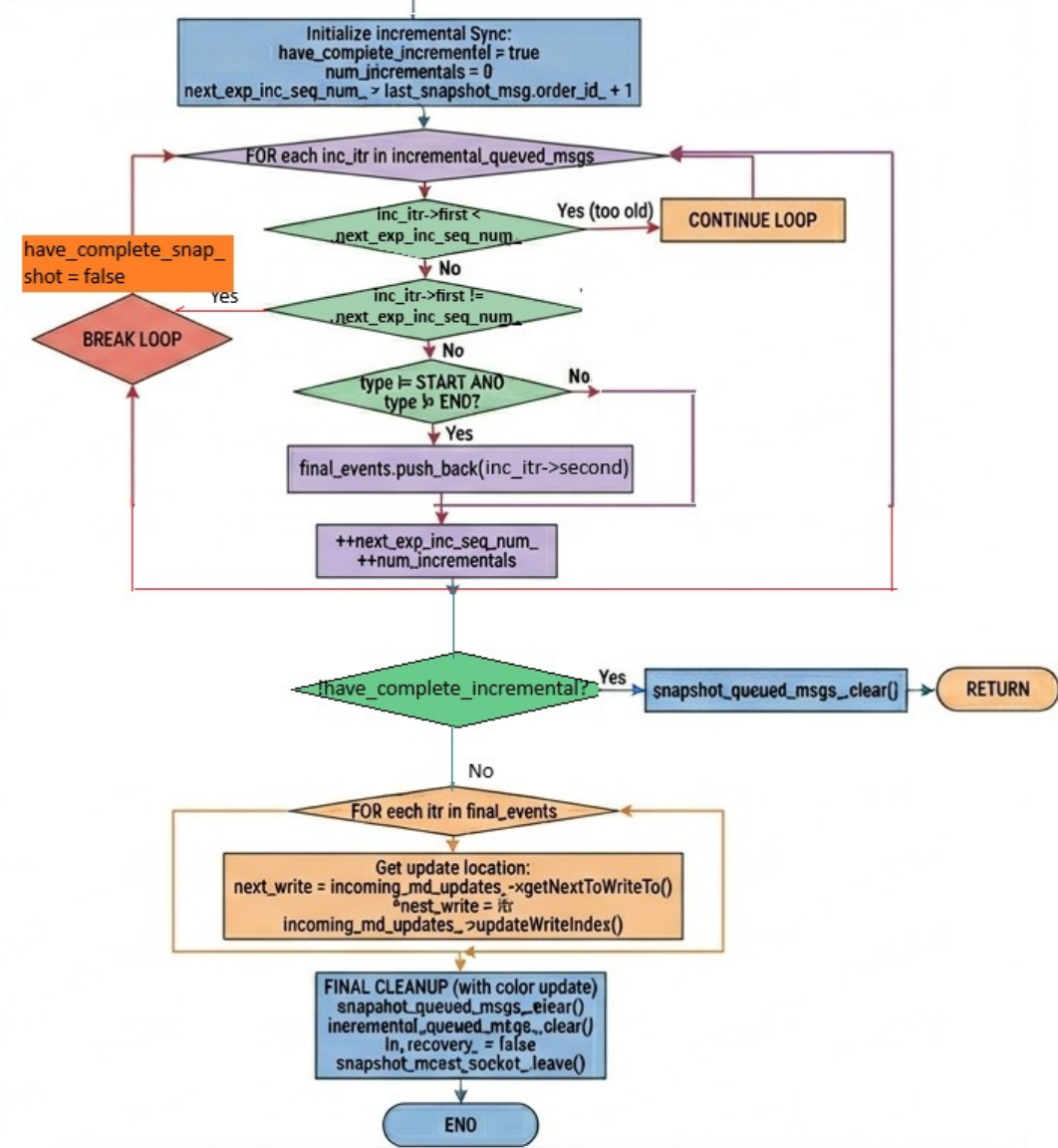
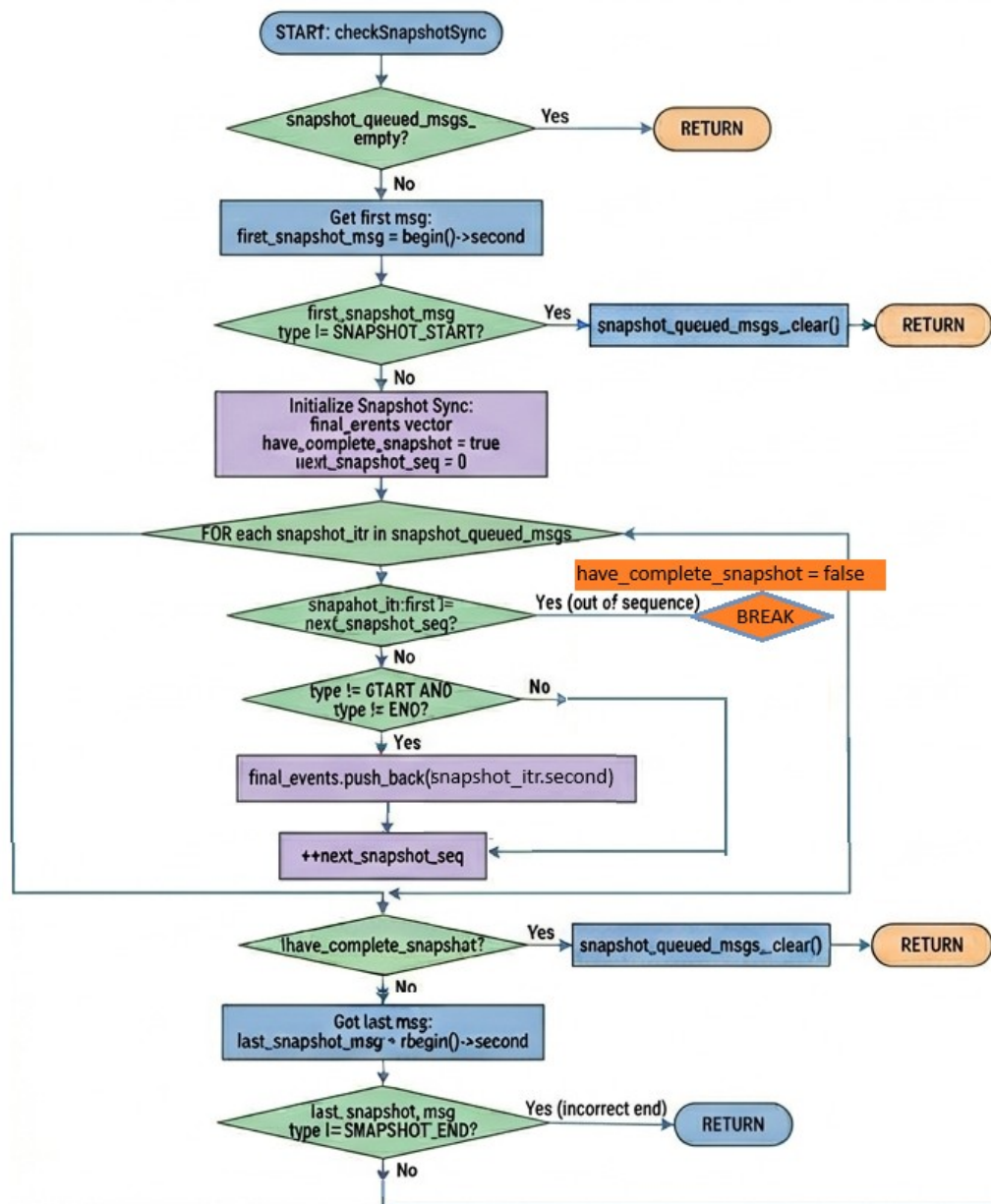


Figure 8.2 – Example state of snapshot and incremental queues when recovery is possible



Part 4: Demo

THANK YOU

Briefly elaborate on what you want to discuss.